
déconvolution d'un signal triangulaire

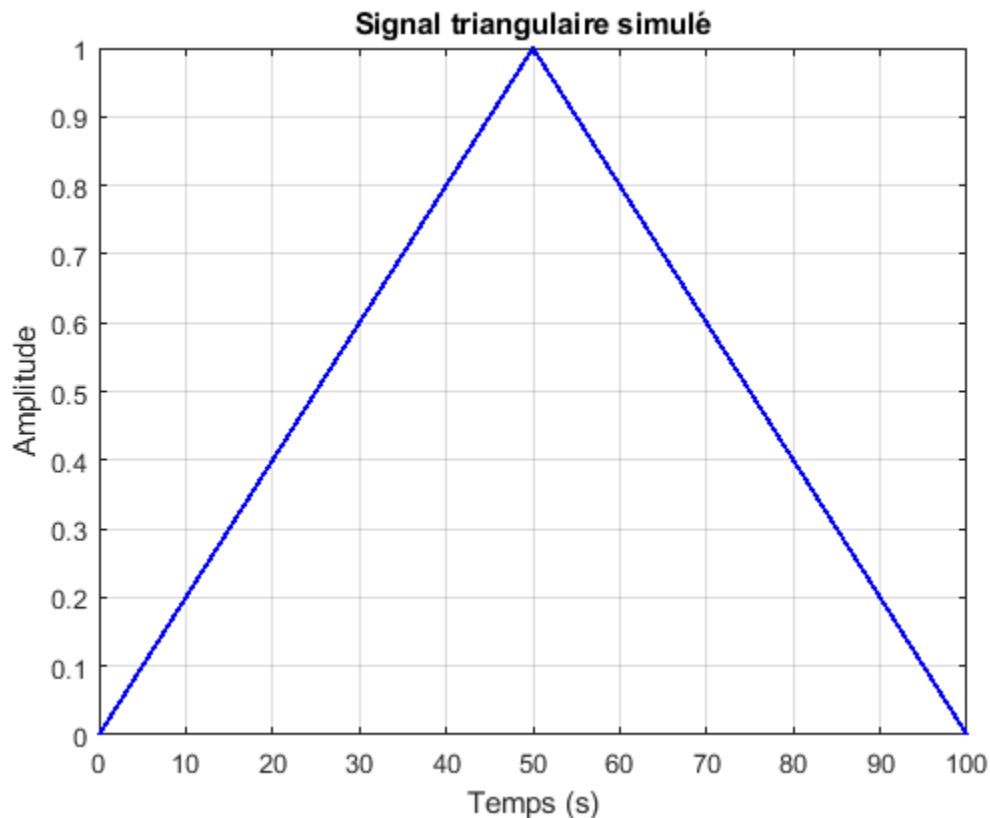
1. Construction d'un jeu données

La fréquence d'échantillonnage est fixée à la valeur $f_e = 1\text{Hz}$. Les échantillons du signal d'entrée sont notés $x[n]$, ceux de la réponse impulsionnelle du filtre $h[n]$ et ceux du signal mesuré en sortie $y[n]$. Le signal mesuré en sortie est sujet à des incertitudes additives notées $b[n]$ et le problème direct peut alors être modélisé comme suit : $y[n] = (x \star h)[n] + b[n]$

a. Simulation du signal d'entrée

Le signal d'entrée, supposé être triangulaire.

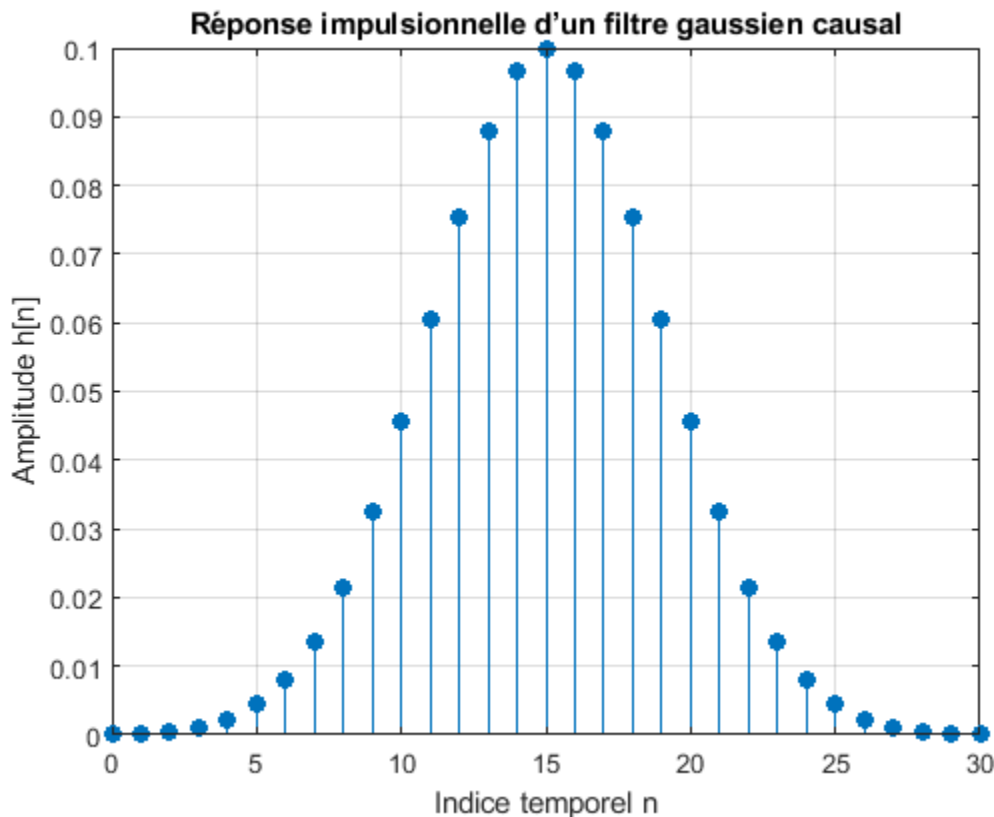
```
L = 100; % duree d' observation du signal d' entree ( secondes )
fe = 1; % frequence d' echantillonnage (Hz)
Te = 1/ fe; % periode d' echantillonnage ( secondes )
Nx = L/Te; % nombre d' echantillons du signal x
nx = 0:Nx; % indice temporel ( entier )
tx = nx*Te; % base de temps ( secondes )
% Signal triangulaire
x1 = (tx(tx <= L/2)) / (L/2); x2 = (-tx(tx > L/2) + L) / (L/2); x = [x1, x2]';
%tracé du signal
figure(1); plot(tx', x, 'b', 'LineWidth', 1.5);
xlabel('Temps (s)'); ylabel('Amplitude'); title('Signal triangulaire
simulé'); grid on;
```



b. Simulation du filtre

Le filtre est de plus supposé causal (i.e. , devaleurs nulles aux temps négatifs) et ses paramètres seront pris égaux respectivement à $m = 15$ et $\sigma = 4$. Enfin, la réponse impulsionnelle est tronquée à un horizon de $N_h = 30$ s.

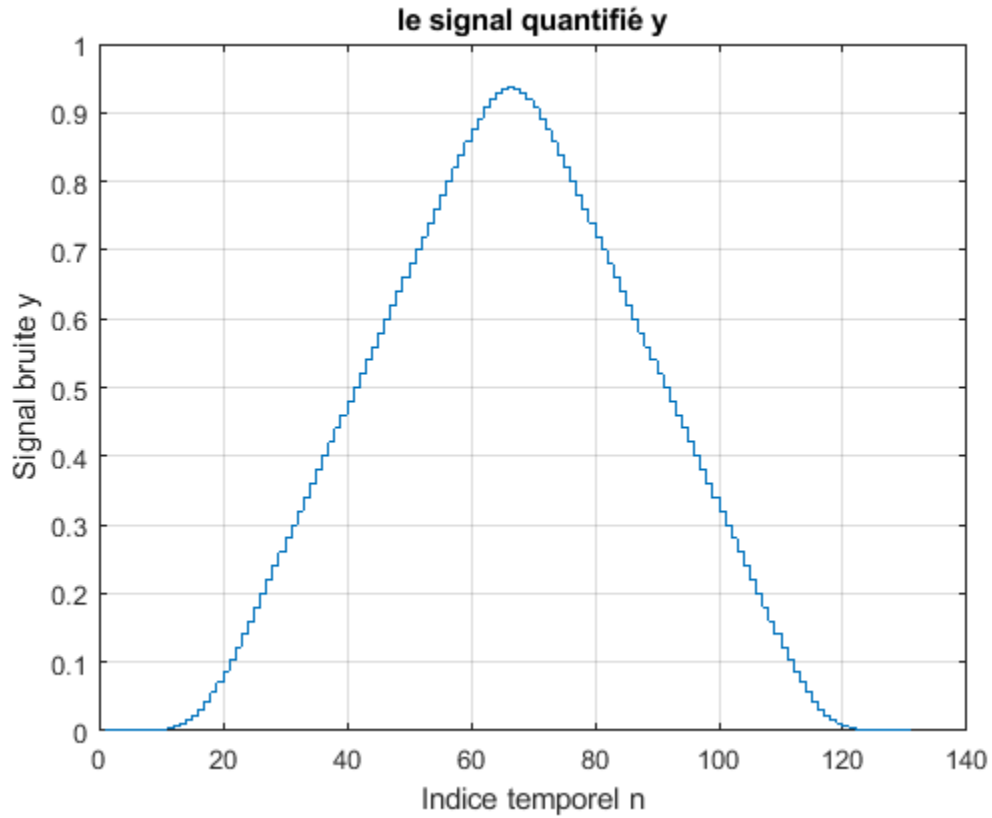
```
sig=4;
m=15;
Nh=30;
n=0:Nh;
h=(1/(sig*sqrt(2*pi)))*exp(-((n-m).^2)/(2*sig^2));
figure(2)
stem(n,h,'filled')
xlabel('Indice temporel n');
ylabel('Amplitude h[n]');
title('Réponse impulsionnelle d'un filtre gaussien causal');
grid on;
```



c.Simulation du problème direct: première approche On calcule la sortie $y_{nb}[n]$ comme le produit de convolution discret à l'aide de la fonction conv

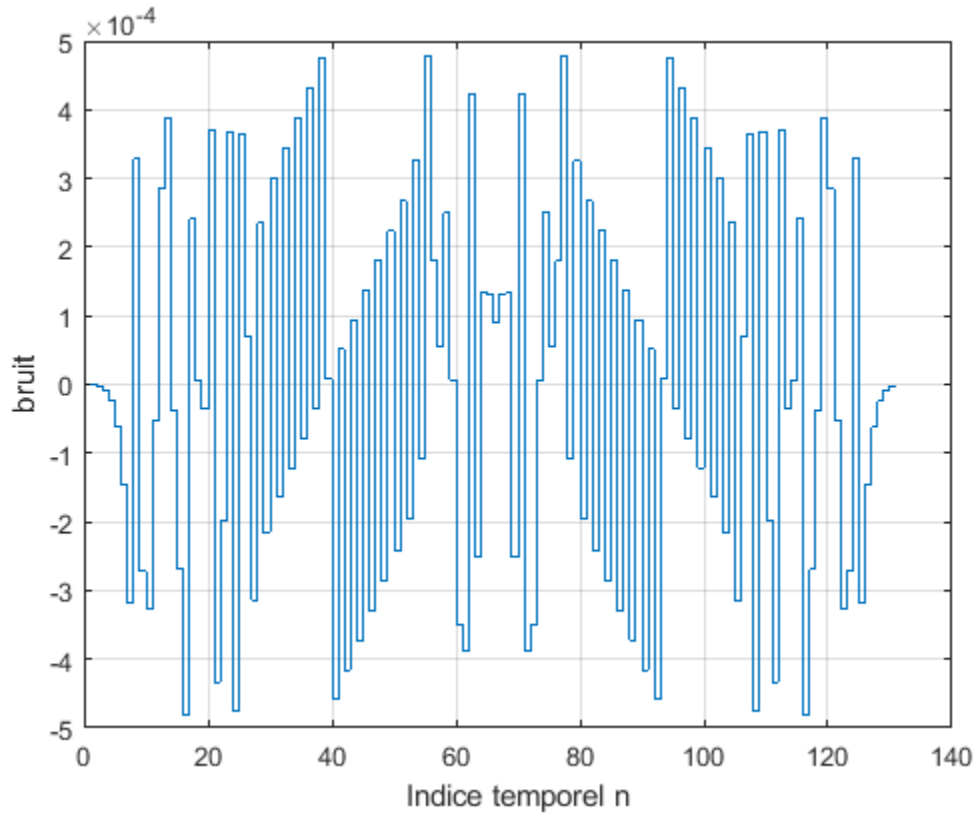
```
% Le signal de sortie y est mesuré à l'aide d'un convertisseur analogique-
numérique, il est sujet
% au bruit de quantification.
yn=conv(x,h); %signal non bruité
y=round(yn*2^10)/2^10;
figure(3)
stairs(y)
```

```
xlabel('Indice temporel n');  
ylabel('Signal bruité y');  
title('le signal quantifié y');  
grid on;
```



le bruit de quantification $b[n]$:

```
b=y-yn;  
figure(4)  
stairs(b) % représentation de bruit  
xlabel('Indice temporel n');  
ylabel('bruit');  
grid on;
```



d. Simulation du problème direct par approche matricielle

Dans cette partie, le signal de sortie non bruité est simulé par l'équation matricielle $y_n = \sum_{k=0}^{N_h-1} x_{n-k} h_k$. L'usage d'un paramétrage soigné est recommandé pour permettre de modifier aisément les conditions de simulation et d'affichage des résultats.

La dimension N_y du vecteur de sortie y_{nb} à partir des dimensions N_x et N_h des vecteurs d'entrée x et de réponse impulsionnelle h .

```
Nx=length(x) % Dimension du vecteur d'entrée x
Nh=length(h) % Dimension du vecteur de réponse impulsionnelle h
Ny=Nx+Nh-1 % Dimension du vecteur de sortie y_nb
```

$N_x =$

101

$N_h =$

31

$N_y =$

131

9. Construction de la matrice de convolution H On initialise le premier vecteur colonne avec des zéros et le premier vecteur ligne. puis de remplacer certains de leurs éléments avec les éléments de h.

```
col = zeros(Ny, 1);
col(1:length(h)) = h; % Insertion de h dans le premier vecteur colonne
lig=[h(1), zeros(1, Nx - 1)]; %la premier vecteur ligne
H=toeplitz(col,lig);
```

La sortie non bruitée par une opération matricielle

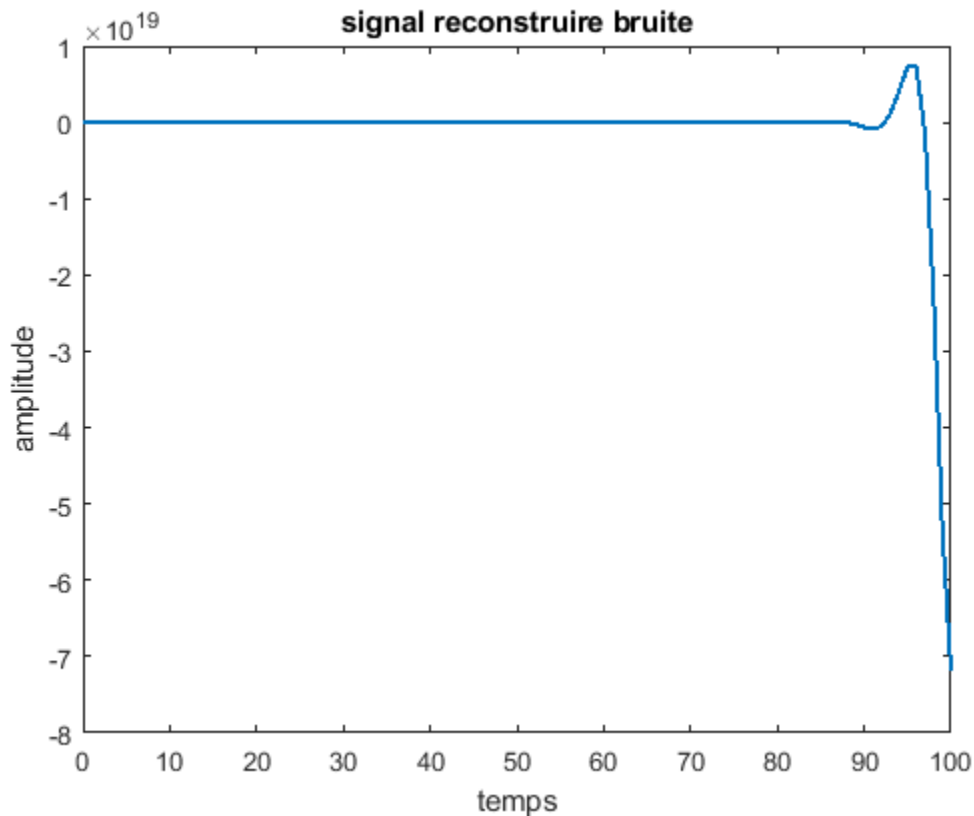
```
Y_nb_toe=H*x;
```

2. Déconvolution

a. Déconvolution « naïve » et constat de l'instabilité

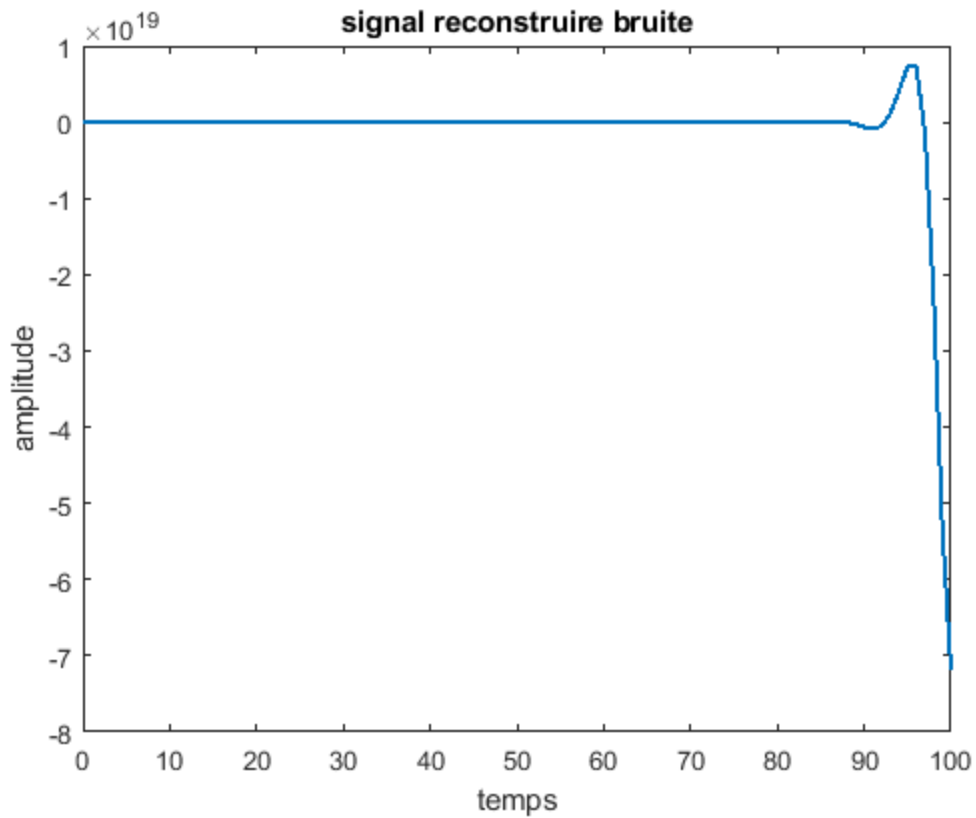
Reconstruire le signal d'entrée en utilisant l'opération de déconvolution dans le cas idéal à la sortie non bruitée et à partir des échantillons mesurés du signal de sortie y (bruité).

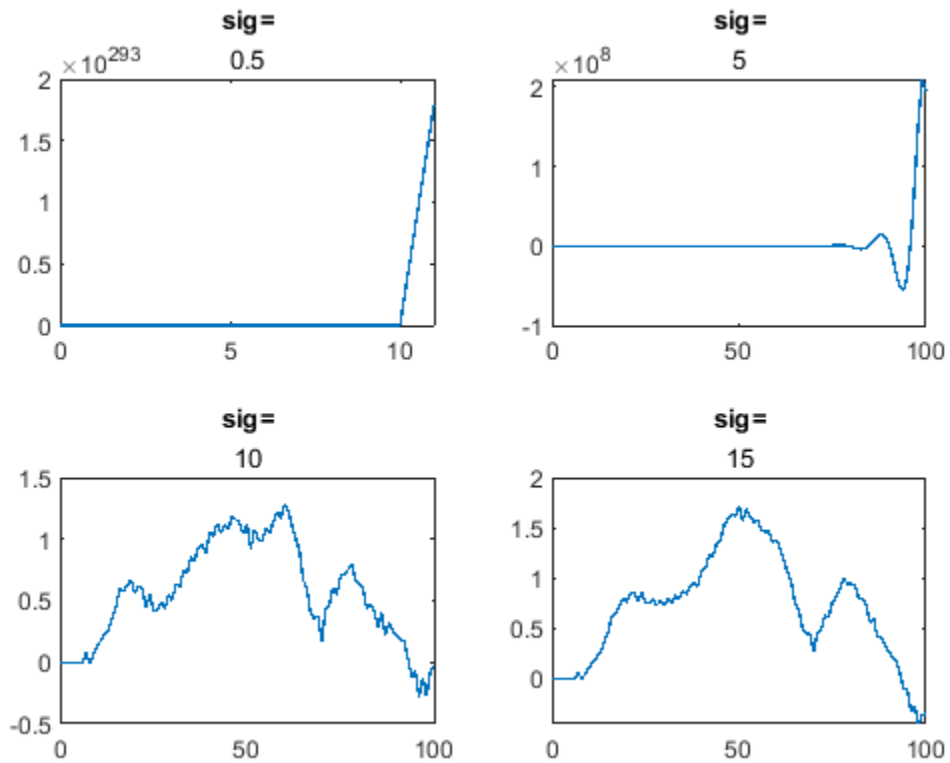
```
xnrec=deconv(yn, h); %11
figure
plot(tx,xnrec,'linewidth',1.5)
title('signal reconstruire non bruité'); xlabel('temps'); ylabel('amplitude')
%12
xrec=deconv(y, h);
figure(5)
plot(tx,xrec,'linewidth',1.5)
title('signal reconstruire bruité'); xlabel('temps'); ylabel('amplitude')
```



l'influence de la « forme » de la réponse impulsionnelle sur la qualité de la reconstruction en modifiant la valeur du paramètre σ .

```
i=1;
figure(6)
for s=[0.5 5 10 15]
    h=(1/(s*sqrt(2*pi)))*exp(-(n-m).^2)/(2*s.^2));
    xrec=deconv(y,h);
    subplot(2,2,i)
    plot(tx,xrec,'linewidth',1); title("sig=",s)
    i=i+1;
end
```

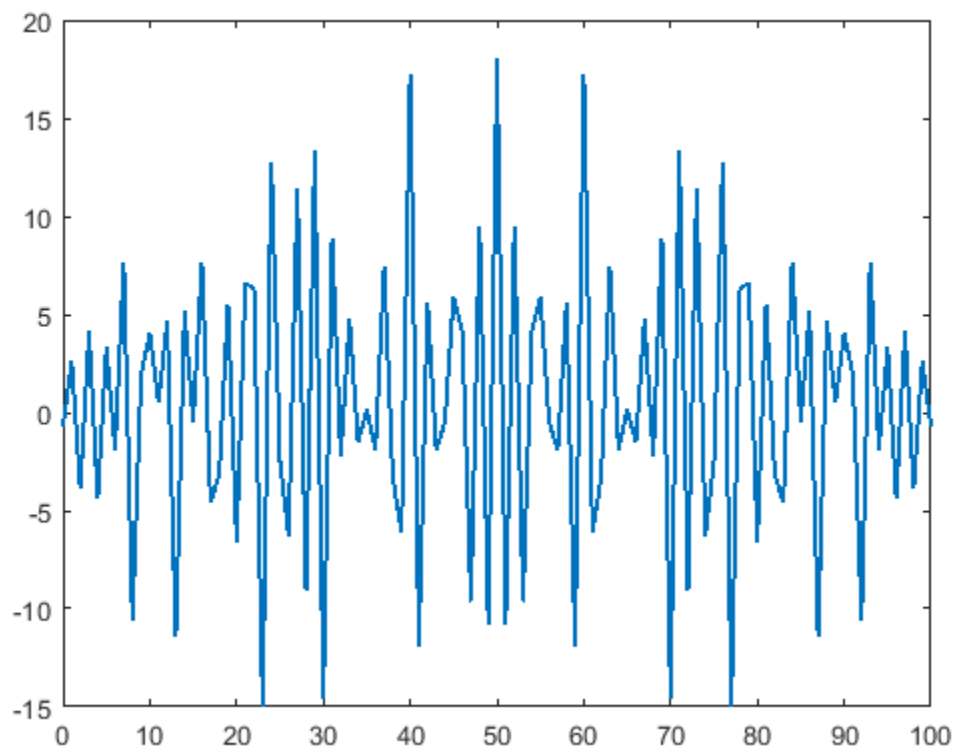




On remarque que plus que sigma grand plus que le signal x reconstruit est plus proche à sa forme triangulaire mais il y a un grand différent.

b. Déconvolution par moindres carrés 14. Calcule le signal déconvolué à l'aide de l'estimateur des moindres carrés

```
xrecM = pinv(H) * y; % Utilisation de la pseudo-inverse pour l'estimation
figure(7)
plot(tx,xrecM,'linewidth',1.5);
```



l'énergie de l'erreur de reconstruction.

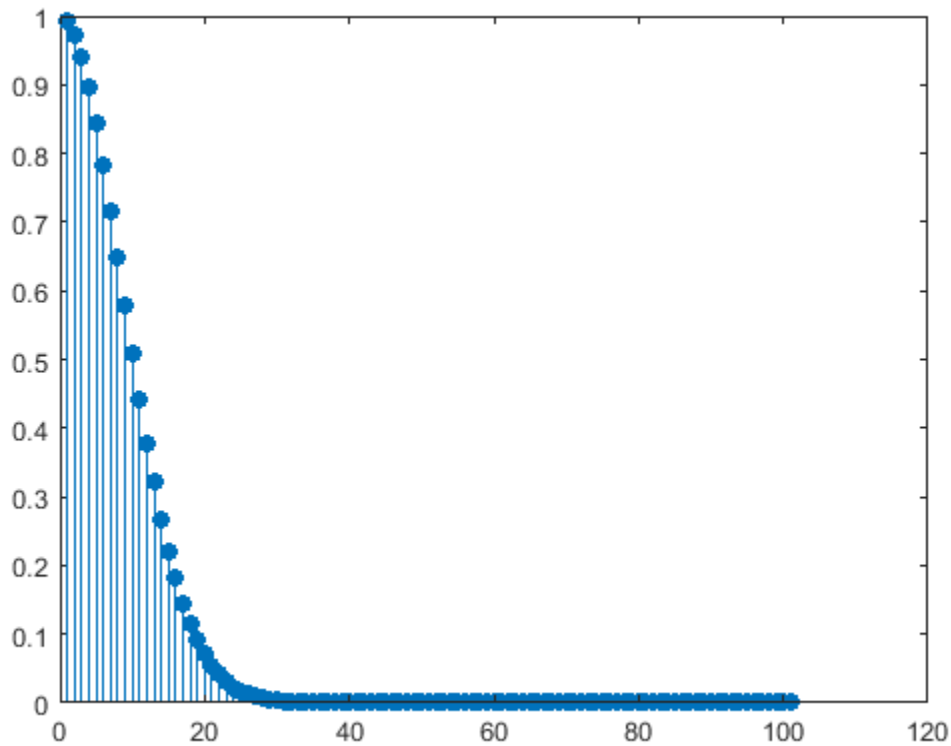
```
err=y-H*xrecM; %calcul de l'erreur
ener=sum(err.^2)
```

ener =

1.7042e-06

les valeurs singulières de la matrice H

```
[U,S,V] = svd(H);
figure(8)
stem(diag(S),'filled')
% Calcul du conditionnement
Smax=max(diag(S));
Smin=min(diag(S));
Cond=Smax/Smin;
```

c. Déconvolution par régularisation linéaire quadratique ($\ell_2\ell_2$) Construction de D1

```
dcoll = zeros (1 , Nx ) ;
dcoll (1:2) = dcoll (1:2) + [1 -1];
dlig1 = zeros (1 , Nx ) ;
dlig1 (1 ,1) = dcoll (1 ,1) ;
D1 = toeplitz ( dcoll , dlig1 ) ;
lambda_vals = logspace(-7, 2, 50);
errors = zeros(size(lambda_vals));

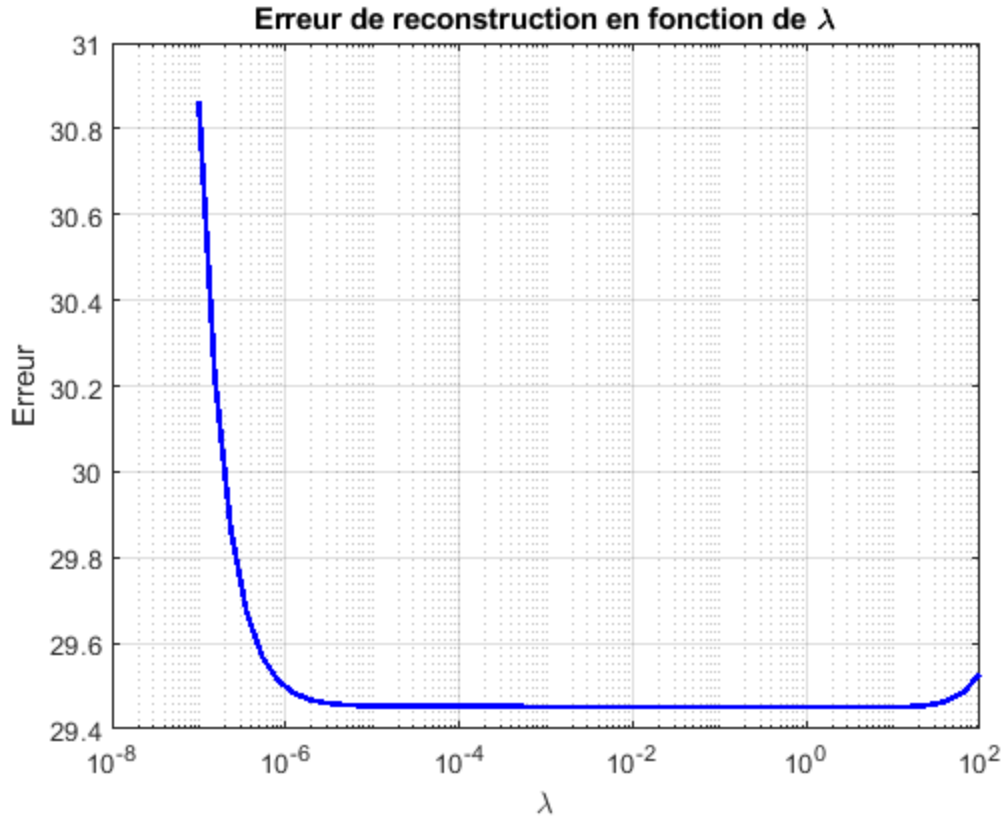
for i = 1:length(lambda_vals)
    lambda = lambda_vals(i);
    x_reg = (H' * H + lambda * (D1' * D1)) \ (H' * y);
    errors(i) = norm(x - x_reg');
end
[~, best_idx] = min(errors);
lambda_opt = lambda_vals(best_idx);
x_reg_opt = (H' * H + lambda_opt * (D1' * D1)) \ (H' * y);

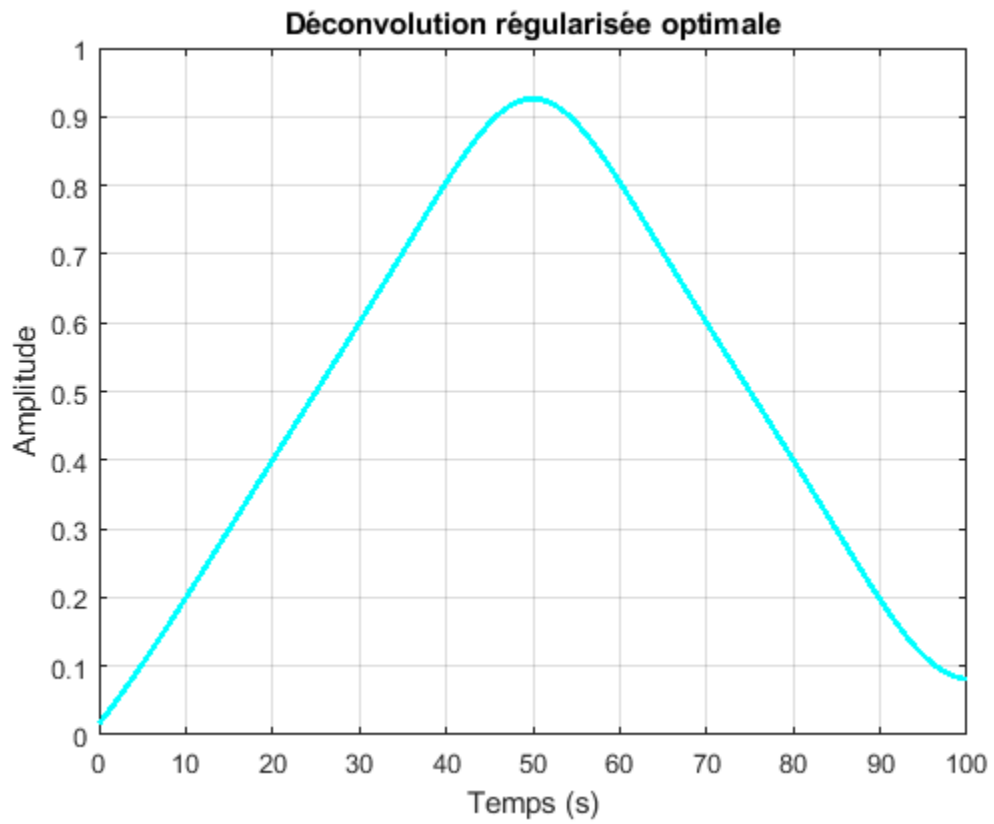
figure; semilogx(lambda_vals, errors, 'b', 'LineWidth', 2);
title('Erreur de reconstruction en fonction de \lambda'); xlabel('\lambda');
ylabel('Erreur'); grid on;

figure; plot(tx, x_reg_opt, 'c', 'LineWidth', 2);
title('Déconvolution régularisée optimale'); xlabel('Temps (s)');
ylabel('Amplitude'); grid on;
```

```
disp(['Valeur optimale de lambda : ', num2str(lambda_opt)]);
```

Valeur optimale de lambda : 7.906





Published with MATLAB® R2024a